

Sabre Site Reliability Engineer I

Campus Drive Preparation Handbook

Complete Interview Preparation Guide for Final Year Students

Center for Technical Training

Version 1.1 | Validated against the Sabre Site Reliability Engineer I job description dated July 2026. Verify all company facts, compensation details and process information with your placement cell before the drive.

Table of Contents

Part 1 — Know The Company.....	5
1.1 Sabre in one paragraph (memorise the substance, not the words).....	5
1.2 Why this matters for an SRE	5
1.3 One-line answer you should have ready.....	5
1.4 Things to check the week of the drive	5
Part 2 — Decoding The Job Description.....	7
2.1 The eligibility gate.....	7
2.2 Every JD responsibility, translated	7
2.3 Requirements, ranked by how much they'll actually test	7
2.4 The warning in the JD — read this twice.....	8
2.5 Possible campus process — confirm with the placement cell	8
2.6 Employment details the JD does not specify — ask, do not assume	8
Part 3 — The 5-Day Preparation Plan	9
3.1 Day 1 — Linux and operating systems.....	9
3.2 Day 2 — Networking and databases	9
3.3 Day 3 — Cloud (AWS and Google Cloud).....	9
3.4 Day 4 — DevOps and SRE concepts.....	9
3.5 Day 5 — Coding, projects, HR and a full mock.....	9
Part 4 — Linux & Operating Systems	10
4.1 [L1] The commands you must know cold	10
4.2 [L1] Permissions — this is a high-probability interview topic.....	10
4.3 [L1] Boot process — a common one-liner question.....	10

4.4 [L1] Core OS theory.....	10
4.5 [L2] Diagnostic scenarios — this is the L1/L2 triage the JD mentions	11
4.6 [L3] Advanced Linux talking points	12
Part 5 — Networking.....	12
5.1 [L1] OSI and TCP/IP	12
5.2 [L1] TCP vs UDP.....	12
5.3 [L1] “What happens when you type a URL and press Enter?”	12
5.4 [L1] DNS record types	13
5.5 [L1] HTTP status codes	13
5.6 [L2] Load balancing.....	13
5.7 [L2] Subnetting essentials	13
5.8 [L3] Advanced	14
Part 6 — Cloud Computing (AWS & GCP).....	14
6.1 [L1] Fundamentals.....	14
6.2 [L1] AWS ↔ GCP service mapping.....	14
6.3 [L1] AWS core services in one line each.....	15
6.4 [L2] Migration — the JD’s first responsibility.....	15
6.5 [L2] Design a highly available architecture	16
6.6 [L3] Advanced	16
6.7 [L1] Google Cloud in depth — the JD names GCP as well as AWS	17
Part 7 — Databases	18
7.1 [L1] SQL fundamentals.....	18
7.2 [L1] Queries you should be able to write on paper	18
7.3 [L2] SQL vs NoSQL	19
7.4 [L2] Reliability-relevant database topics	19
7.5 [L3] Advanced	19
Part 8 — DevOps Tooling.....	20
8.1 [L1] What DevOps actually is.....	20
8.2 [L1] Git	20
8.3 [L1] CI/CD.....	20
8.4 [L1] Containers.....	21
8.5 [L2] Kubernetes.....	21
8.6 [L2] Infrastructure as Code and configuration management.....	21
8.7 [L2] Monitoring and observability.....	22

8.8 [L1] Security fundamentals every SRE needs	23
Part 9 — SRE Core Concepts	23
9.1 [L1] What is SRE?	23
9.2 [L1] SLI, SLO, SLA — you will be asked, so be exact	24
9.3 [L1] Error budget	24
9.4 [L1] Toil — the JD names this explicitly	24
9.5 [L1] Reliability metrics	25
9.6 [L1] Incident management — the JD’s “high-pressure situations”	25
9.7 [L2] Reliability design patterns	25
9.8 [L2] On-call	26
9.9 [L3] Advanced	26
Part 10 — Scripting & Automation	27
10.1 [L1] Bash	27
10.2 [L1] Python for ops	28
10.3 [L2] DSA — how much you actually need	28
Part 11 — Your Resume & Projects	30
11.1 The rules	30
11.2 If you have no SRE experience (most of you)	30
11.3 Project answer template (STAR)	30
Part 12 — The Question Bank	31
12.1 Linux (25)	31
12.2 Networking (20)	31
12.3 Cloud (25)	32
12.4 DevOps (25)	32
12.5 SRE-specific (25)	33
12.6 Scenario questions (15) — where L2 candidates are separated	34
12.7 Sample model answers for the four highest-stakes questions	34
Part 13 — HR & Behavioural Round	36
13.1 The standard set	36
13.2 The four the JD specifically sets up	36
13.3 “Tell me about yourself” — a working structure	36
13.4 Weakness — how to answer without self-sabotage	36
13.5 Questions to ask them	37
13.6 Practical checklist	37

Part 14 — Mock Interview Structure.....	37
14.4 Practical readiness assessment — do this, do not just read it.....	38

Part 1 — Know The Company

1.1 Sabre in one paragraph (memorise the substance, not the words)

Sabre is a US travel technology company headquartered in **Southlake, Texas**, with roughly **3,700 employees across ~60 countries**, and over **60 years** in the industry. It sits in the middle of the travel ecosystem as a **GDS — Global Distribution System**. On one side are **travel suppliers** (airlines, hotels, car rental companies, railways, tour operators); on the other are **travel buyers** (travel agencies, travel management companies, corporate travel desks). Sabre's platforms let those two sides find each other, price inventory, book it, and service it. The product lines include data-driven business intelligence, mobile, distribution, and SaaS solutions.

1.2 Why this matters for an SRE

This context is not trivia — it is the reason the SRE role exists at Sabre:

- **Airline reservation systems are 24×7 mission-critical.** Booking and reservation platforms are expected to be available continuously, and outages affect many downstream travel businesses at once. Treat the commercial impact as significant rather than quoting a specific figure.
- **Extreme, spiky traffic.** Travel demand is seasonal and event-driven, and disruptions such as weather or schedule changes create sudden load. Capacity planning and autoscaling matter in this kind of environment.
- **Decades-old legacy systems.** Sabre has operated travel technology platforms for over 60 years, so parts of the estate are long-established datacenter systems. That is exactly why the JD's first bullet is *"migrate legacy/datacenter deployed applications to a multi cloud environment (AWS, Google Cloud)."*
- **Global footprint.** Multi-region and latency-sensitive, operating across roughly 60 countries, so data-residency and regulatory considerations vary by market.

1.3 One-line answer you should have ready

"Sabre is a travel technology company that connects airlines, hotels and other suppliers with travel agencies and corporate buyers through its distribution and SaaS platforms. What interested me about the SRE role is that these are 24×7 transaction-heavy systems being migrated from datacenters to AWS and GCP — so reliability engineering there is a genuinely hard problem, not a maintenance job."

That single sentence answers "What do you know about us?" and "Why this role?" simultaneously. Practice it until it sounds natural, not recited.

1.4 Things to check the week of the drive

- Sabre's recent news, quarterly results, and any cloud partnership announcements (Google Cloud has historically been significant for Sabre).
- The Sabre careers page and LinkedIn for the Bengaluru office and the SRE team.
- Any Sabre engineering blog posts on migration, observability, or resilience.

(Verify current facts yourself before the drive — this handbook was written from the JD, and company news moves.)

Part 2 — Decoding The Job Description

2.1 The eligibility gate

Criterion	Requirement
Degree	BE / B.Tech — CSE, IT, ISE, EEE, ECE
Year of passing	2027
Marks	7.0 CGPA or above in all declared semesters, as on the hiring event date
Backlogs	No active backlogs on the hiring event date
Location	Bengaluru
Stipend	INR 40,000 per month
CTC	INR 9,41,460 p.a. (includes 5% variable)
Joining bonus	INR 2,00,000, paid in two instalments
Travel	Must be willing to travel

Do this before the drive: print your semester-wise CGPA sheet, confirm every semester is ≥ 7.0 , and confirm zero active backlogs. Carry originals plus two photocopy sets, ID proof, and multiple copies of your resume.

2.2 Every JD responsibility, translated

JD says	Likely areas they may test
Migrate legacy/datacenter apps to multi-cloud (AWS, GCP)	IaaS vs PaaS vs SaaS, VMs vs containers, lift-and-shift vs re-architecture, VPC/subnets, migration risks and rollback
Own end-to-end responsibility for production apps	What “ownership” means, run vs build, on-call, runbooks, change management
CI/CD and DevOps tooling for faster, stable deployments	Git, Jenkins/GitLab CI, Docker, Kubernetes, Ansible, Terraform, pipeline stages, rollback strategies
Being part of high-pressure incidents and driving closure	Incident lifecycle, severity levels, comms, blameless postmortem, RCA
Reduce toil by automating manual processes	Definition of toil, scripting in Bash/Python, “what would you automate first?”
Improve stability, reduce MTTR	SLI/SLO/SLA, error budget, MTTR/MTTD/MTBF, monitoring vs observability
L1 and L2 diagnostic and triage per SOPs	Linux troubleshooting commands, log reading, a systematic debugging method
Network architectures and databases	OSI/TCP-IP, DNS, HTTP status codes, load balancing, SQL vs NoSQL, indexes, replication

2.3 Requirements, ranked by how much they’ll actually test

1. **Understanding of Cloud** — stated without “good to have.” This is the hard requirement. Do not go in shaky on cloud basics.

2. **Linux** — marked “good to have,” but in practice it is the backbone of every SRE interview. Treat it as mandatory.
3. **DevOps tooling** — “good to have.” Concept-level clarity plus one hands-on project is plenty for a fresher.
4. **Multi-tasking, English communication, willingness to travel, attention to detail** — these are graded in the HR round and in *how* you answer technical questions, not in a separate test.

2.4 The warning in the JD — read this twice

Sabre states plainly: only students genuinely interested in an SRE career should apply, students who intend to move into development should not apply, and they do **not** consider candidates for other roles during or after the process.

What this means for you tactically:

- Never say “I want to move into development later,” “SRE is a good entry point,” or “I’ll switch teams after two years.” Any of these can seriously weaken or end your candidature.
- Have a real, specific reason for wanting SRE. Weak: *“I like operations.”* Strong: *“I like the debugging side more than the feature side. In my project I spent more time figuring out why the deployment kept failing than writing the app, and I found that more satisfying. SRE is that work, done properly, with automation.”*
- If you are genuinely only interested in development — be honest with yourself and skip this drive. Taking a seat you’ll abandon hurts you and your juniors.

2.5 Possible campus process — confirm with the placement cell

Expect some subset of: online aptitude + technical MCQ → technical round 1 (fundamentals) → technical round 2 (scenario/depth) → managerial → HR. Some drives add a group discussion or a communication assessment. Confirm the exact structure with your placement cell; prepare as though all rounds exist.

2.6 Employment details the JD does not specify — ask, do not assume

The JD gives the stipend, CTC, variable component and joining bonus. It does **not** state the internship duration, the conditions for conversion to full-time, the joining date, the dates of the two joining-bonus instalments, how the 5% variable is calculated, or the shift and on-call structure. Get these confirmed by your placement cell or Sabre HR rather than assuming them, and do not quote assumed figures back to the interviewer.

Part 3 — The 5-Day Preparation Plan

Five days is tight but sufficient if you are disciplined. Roughly 8 focused hours a day. Do not read passively — type the commands, break things, and explain concepts aloud.

3.1 Day 1 — Linux and operating systems

3.2 Day 2 — Networking and databases

3.3 Day 3 — Cloud (AWS and Google Cloud)

3.4 Day 4 — DevOps and SRE concepts

3.5 Day 5 — Coding, projects, HR and a full mock

Part 4 — Linux & Operating Systems

4.1 [L1] The commands you must know cold

Files and navigation: ls -ltr, cd, pwd, find, locate, cp, mv, rm, mkdir, touch, ln -s, file, stat

Reading files: cat, less, head, tail -f, grep -i -r -n -v -E, awk, sed, cut, sort, uniq -c, wc -l, tr, diff

Processes: ps aux, ps -ef, top, htop, kill, kill -9, pkill, nice, renice, jobs, fg, bg, nohup, &

Disk and memory: df -h, du -sh *, free -m, lsblk, mount, umount, fdisk -l

Performance: vmstat, iostat, sar, uptime, lsof

Network: ip a, ifconfig, netstat -tulpn, ss -tulpn, ping, traceroute, dig, nslookup, curl, wget, telnet, tcpdump, route -n

Users and permissions: useradd, usermod, passwd, groups, id, chmod, chown, chgrp, umask, sudo, su

Services and logs: systemctl start|stop|status|enable, journalctl -u <svc>, /var/log/messages, /var/log/syslog, dmesg

Packages: yum/dnf, apt, rpm -qa

Archives and transfer: tar -czvf, tar -xzvf, gzip, scp, rsync, ssh

4.2 [L1] Permissions — this is a high-probability interview topic

Three permission sets: owner, group, others. Three bits each: read (4), write (2), execute (1).

- 755 = owner rwx, group r-x, others r-x — typical for directories and executables.
- 644 = owner rw-, group r-, others r- — typical for files.
- chmod u+x script.sh — symbolic form.
- On a **directory**: read = list contents, write = create/delete entries, execute = enter/traverse it.

Special bits [L2]: SUID (run as file owner — passwd is the classic example), SGID (run as group / inherit group in directories), sticky bit (only the owner can delete their files — /tmp uses this).

4.3 [L1] Boot process — a common one-liner question

BIOS/UEFI → POST → bootloader (GRUB) → kernel loaded into memory → initramfs → init/systemd → target/runlevel reached → login prompt.

4.4 [L1] Core OS theory

- **Process vs thread:** a process has its own address space; threads share the address space of their process. Threads are cheaper to create and switch between; processes are better isolated.
- **Context switch:** the CPU saves the current process's state (registers, program counter) and loads another's. It costs time and produces no useful work — pure overhead.

- **Scheduling algorithms:** FCFS, SJF, Round Robin, Priority, Multilevel Queue. Know how each affects turnaround and waiting time.
- **Deadlock — four necessary conditions:** mutual exclusion, hold and wait, no preemption, circular wait. Break any one and deadlock cannot occur. Handling: prevention, avoidance (Banker's algorithm), detection and recovery, or ignore (the ostrich approach).
- **Virtual memory and paging:** memory is divided into fixed-size pages mapped to physical frames, so processes can use more memory than physically exists. Segmentation divides by logical unit instead. **Thrashing** is when the system spends more time paging than executing.
- **Page replacement:** FIFO, LRU, Optimal. Belady's anomaly applies to FIFO.
- **IPC:** pipes, named pipes, message queues, shared memory, sockets, signals.
- **Race condition and critical section:** protected with mutexes, semaphores, monitors.
- **Kernel space vs user space:** system calls are the controlled gateway between them.

4.5 [L2] Diagnostic scenarios — this is the L1/L2 triage the JD mentions

“The server is slow. What do you do?” Give a method, not a random command list:

1. uptime — check load average against core count (nproc).
2. top / htop — is it CPU, memory, or I/O wait? Which process?
3. free -m — is memory exhausted? Is swap being used heavily?
4. iostat -x 1 — high %util or await means disk is the bottleneck.
5. df -h and df -i — full disk *or* exhausted inodes.
6. netstat -tulpn / ss -s — connection floods, TIME_WAIT buildup.
7. Application logs — tail -f, grep -i error.
8. Recent changes — was anything deployed, patched, or config-changed? Most incidents follow a change.

Then state the SRE instinct out loud: *“In parallel I'd communicate status and consider mitigation — restart, scale out, or roll back — before doing full root cause analysis. Restore service first, diagnose second.”* That sentence alone marks you as SRE-minded.

“Disk is 100% full.”

df -h to find the filesystem → du -sh /* | sort -h to walk down to the big directory → usually logs. Check for deleted-but-open files with lsof | grep deleted (space is not freed until the process releases the handle). Compress or rotate logs, then fix root cause with logrotate. Never blindly rm -rf.

“A process is consuming 100% CPU.”

Identify with top, confirm with ps -ef | grep <pid>, inspect with strace -p <pid> or thread dump if it's a JVM, check whether it's a legitimate load spike or a runaway loop. Renice as a stopgap; kill only with an understanding of the impact.

“I can't SSH into the server.”

Is it a network problem (ping), a service problem (port 22 — telnet host 22), an auth problem (key permissions, wrong user), a firewall problem (security group / iptables), or is the host itself full/out of memory so sshd can't fork? Work outside-in.

4.6 [L3] Advanced Linux talking points

systemd unit files and dependencies · cgroups and namespaces (the primitives Docker is built on) · ulimit and file descriptor exhaustion · the OOM killer and /proc/<pid>/oom_score · /proc and /sys as kernel interfaces · nftables/iptables chains · SELinux enforcing vs permissive · kernel tuning via sysctl.

Part 5 — Networking

5.1 [L1] OSI and TCP/IP

OSI Layer	Function	Examples
7 Application	User-facing protocols	HTTP, FTP, DNS, SMTP, SSH
6 Presentation	Encoding, encryption, compression	TLS/SSL, JPEG
5 Session	Session establishment and teardown	NetBIOS, RPC
4 Transport	End-to-end delivery, ports	TCP, UDP
3 Network	Logical addressing, routing	IP, ICMP, OSPF, BGP
2 Data Link	Framing, MAC addressing	Ethernet, ARP, switches
1 Physical	Bits on the wire	Cables, NICs, hubs

TCP/IP collapses these into four: Application, Transport, Internet, Network Access.

5.2 [L1] TCP vs UDP

TCP is connection-oriented, reliable, ordered, flow- and congestion-controlled, with a larger header — used for HTTP, SSH, database connections. UDP is connectionless, unreliable, unordered, and fast — used for DNS queries, DHCP, VoIP and streaming, and by some telemetry agents — though many current metrics and logging agents ship over TCP, HTTP or gRPC.

Three-way handshake: SYN → SYN-ACK → ACK. **Four-way teardown:** FIN → ACK → FIN → ACK. Know why TIME_WAIT exists (to absorb delayed duplicate packets) and why thousands of TIME_WAIT sockets on a busy server are normal, not automatically a bug.

5.3 [L1] “What happens when you type a URL and press Enter?”

The single most reliably asked networking question. Structure your answer:

1. Browser checks its cache, then OS cache, then /etc/hosts.
2. DNS resolution: recursive resolver → root → TLD → authoritative nameserver returns the IP (UDP 53, TCP for large responses).

3. ARP resolves the next-hop MAC address on the local network.
4. TCP three-way handshake to the server IP on port 80/443.
5. TLS handshake if HTTPS — cipher negotiation, certificate validation against a trusted CA, session key established.
6. HTTP request sent; it may hit a CDN, a load balancer, a reverse proxy, then an application server.
7. Application queries cache/database, builds the response.
8. HTTP response returns with a status code; browser parses HTML, fetches assets, renders.
9. Connection kept alive or closed.

Mentioning the CDN, load balancer, and reverse proxy hops is what moves this from an L1 to an L2 answer.

5.4 [L1] DNS record types

A (IPv4), AAAA (IPv6), CNAME (alias), MX (mail), NS (nameserver), TXT (verification, SPF/DKIM), PTR (reverse lookup), SOA (zone authority). Understand **TTL** and why it matters during a migration or failover — a long TTL means your DNS change takes hours to propagate, which is a real incident risk.

5.5 [L1] HTTP status codes

- **2xx success:** 200 OK, 201 Created, 204 No Content
- **3xx redirect:** 301 permanent, 302 temporary, 304 Not Modified
- **4xx client error:** 400 Bad Request, 401 Unauthorized (not authenticated), 403 Forbidden (authenticated, not permitted), 404 Not Found, 429 Too Many Requests
- **5xx server error:** 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout

Interview favourite: “What’s the difference between 502 and 504?” 502 means the upstream returned an invalid response; 504 means the upstream did not respond in time. Both are seen at a proxy or load balancer and usually indicate a problem with the upstream dependency — but always also check the proxy’s own capacity, timeout configuration and network path before concluding that.

5.6 [L2] Load balancing

- **L4** balances on IP and port — fast, protocol-agnostic. **L7** balances on HTTP content — path, host header, cookies — enabling routing rules and TLS termination.
- **Algorithms:** round robin, weighted round robin, least connections, IP hash, least response time.
- **Health checks** remove unhealthy backends automatically. **Sticky sessions** pin a user to one backend — convenient but harmful to scalability and failover; stateless services are better.

5.7 [L2] Subnetting essentials

Private ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16. A /24 gives 256 addresses, 254 usable (minus network and broadcast); in AWS, 251 usable because AWS reserves five. Know how to say why you’d split a VPC into public and private subnets across availability zones.

5.8 [L3] Advanced

BGP and routing between autonomous systems · anycast and how CDNs use it · TCP slow start and congestion control · MTU and fragmentation · HTTP/2 multiplexing vs HTTP/1.1 head-of-line blocking · QUIC/HTTP3 over UDP · TLS 1.3 handshake reduction · mTLS · WebSockets vs long polling · service mesh (Istio, Envoy) and sidecar proxies.

Part 6 — Cloud Computing (AWS & GCP)

This is the one requirement Sabre did *not* mark “good to have.” Give it the most time.

6.1 [L1] Fundamentals

- **Cloud computing:** on-demand delivery of compute, storage, networking and services over the internet, paid for by usage.
- **Service models:**
 - **IaaS** — you get raw infrastructure; you manage OS and above. EC2, Compute Engine.
 - **PaaS** — you deploy code; the platform manages runtime and OS. Elastic Beanstalk, App Engine.
 - **SaaS** — you consume finished software. Gmail, Salesforce — and much of Sabre’s own portfolio.
- **Deployment models:** public, private, hybrid, multi-cloud (Sabre is explicitly multi-cloud: AWS + GCP).
- **Five characteristics:** on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service.
- **Region vs Availability Zone:** a region is a geographic area; an AZ is one or more discrete datacenters within it with independent power and networking. **Deploy across multiple AZs for high availability; across multiple regions for disaster recovery.**
- **Scalability vs elasticity:** scalability is the ability to grow; elasticity is automatic growth *and shrink* with demand. **Vertical scaling** = bigger machine (has a ceiling, usually needs a restart); **horizontal scaling** = more machines (preferred, needs stateless design).
- **Shared responsibility model:** the provider secures *of* the cloud (hardware, hypervisor, physical security); you secure *in* the cloud (data, IAM, OS patching, network config, encryption). Say this crisply — it’s a very common question.

6.2 [L1] AWS ↔ GCP service mapping

Function	AWS	Google Cloud
Virtual machines	EC2	Compute Engine
Object storage	S3	Cloud Storage
Block storage	EBS	Persistent Disk
Serverless functions	Lambda	Cloud Run functions

Managed Kubernetes	EKS	GKE
Containers (managed)	ECS / Fargate	Cloud Run
Relational database	RDS / Aurora	Cloud SQL / AlloyDB / Spanner
NoSQL	DynamoDB	Firestore / Bigtable
Networking	VPC	VPC
Load balancing	ELB / ALB / NLB	Cloud Load Balancing
DNS	Route 53	Cloud DNS
CDN	CloudFront	Cloud CDN
Identity	IAM	Cloud IAM
Monitoring	CloudWatch	Cloud Monitoring (Ops Suite)
IaC	CloudFormation	Infrastructure Manager / Terraform
Data warehouse	Redshift	BigQuery
Messaging	SQS / SNS	Pub/Sub
Secrets	Secrets Manager	Secret Manager

Knowing both columns is directly relevant here — the JD names both clouds.

6.3 [L1] AWS core services in one line each

- **EC2** — resizable virtual machines. Know instance families, on-demand vs reserved vs spot, AMIs, user data.
- **S3** — object storage with 11 nines of durability. Buckets, keys, storage classes (Standard, IA, Glacier), versioning, lifecycle policies, bucket policies. **S3 is not a filesystem.**
- **EBS** — block storage normally attached to a single EC2 instance; snapshots for backup. Supported io1/io2 Provisioned IOPS volumes can use Multi-Attach across several compatible instances in the same Availability Zone.
- **VPC** — your isolated network: subnets, route tables, internet gateway, NAT gateway, security groups (stateful, instance-level, allow-only) and NACLs (stateless, subnet-level, allow and deny). Know that difference — it is asked constantly.
- **IAM** — users, groups, roles, policies. **Principle of least privilege.** Use roles for EC2, never embed access keys.
- **ELB** — ALB (L7), NLB (L4), CLB (legacy).
- **Auto Scaling Group** — launch template, min/desired/max, scaling policies, health checks.
- **RDS** — managed relational DB, Multi-AZ for HA, read replicas for read scaling.
- **CloudWatch** — metrics, logs, alarms, dashboards.
- **Route 53** — DNS with health checks and routing policies (weighted, latency, failover, geolocation).

6.4 [L2] Migration — the JD’s first responsibility

The 7 R’s of migration (current AWS guidance):

1. **Rehost** (“lift and shift”) — move as-is. Fast, low risk, no cloud-native benefit.

2. **Relocate** — move a whole hosting environment (for example VMware workloads) to the cloud without changing the applications themselves.
3. **Replatform** — small optimisations, e.g. move the database to RDS.
4. **Repurchase** — replace with a SaaS product.
5. **Refactor / Re-architect** — redesign cloud-native (microservices, containers, serverless). Highest effort, highest payoff.
6. **Retire** — decommission what nobody uses.
7. **Retain** — leave it where it is for now.

Note: some older interview material still refers to “the 6 R’s.” Current AWS guidance uses seven, adding Relocate. Either answer is acceptable if you can name the strategies and say when each applies.

Migration phases: discovery and inventory → dependency mapping → assessment and R-selection → design target architecture → pilot with a low-risk app → data migration → cutover → validate → optimise → decommission.

Risks to name if asked: data loss, extended downtime, hidden dependencies, latency between migrated and non-migrated components, cost overrun, licensing, security and compliance gaps, and lack of rollback. **Always have a rollback plan and run a pilot first.**

Why multi-cloud (Sabre’s exact situation): avoid vendor lock-in, use each provider’s strengths, meet data residency requirements across countries, improve resilience, and gain commercial leverage. **Trade-offs:** operational complexity, duplicated tooling, cross-cloud data egress cost, and a broader skill requirement per engineer.

6.5 [L2] Design a highly available architecture

If asked to whiteboard it, cover: Route 53 → CloudFront/CDN → ALB across ≥ 2 AZs → Auto Scaling group of stateless app servers in **private** subnets → RDS Multi-AZ with read replicas → ElastiCache for hot reads → S3 for static assets → a NAT gateway in each Availability Zone for outbound traffic, with each private subnet routing to the NAT gateway in its own AZ (a single zonal NAT gateway creates an AZ dependency) → security groups between tiers → CloudWatch alarms and centralised logging → backups and a tested DR plan. State your **RTO and RPO** targets — that shows maturity.

6.6 [L3] Advanced

Well-Architected Framework pillars (operational excellence, security, reliability, performance efficiency, cost optimisation, sustainability) · serverless architecture and cold starts · spot instance interruption handling · cost optimisation techniques (rightsizing, savings plans, storage tiering) · Transit Gateway and VPC peering · PrivateLink · landing zones and multi-account strategy · immutable infrastructure.

6.7 [L1] Google Cloud in depth — the JD names GCP as well as AWS

Most campus preparation is AWS-heavy. Because the JD names both clouds, be able to hold a conversation in Google Cloud vocabulary too. You do not need certification depth — you need to know what each service is for and how it maps to the AWS equivalent you already learnt.

- **Compute Engine** — virtual machines. Machine families and custom machine types, managed instance groups (the equivalent of an Auto Scaling group), instance templates, live migration during host maintenance, and preemptible/Spot VMs for cheap interruptible work.
- **Google Kubernetes Engine (GKE)** — managed Kubernetes. Know the difference between Standard mode (you manage node pools) and Autopilot mode (Google manages nodes and you pay per pod).
- **Cloud Run** — runs stateless containers with request-based autoscaling, including scale to zero. Cloud Run functions is the current form of Google's event-driven functions offering. Use Cloud Run when you have a container and do not want to manage a cluster.
- **VPC** — note the real difference from AWS: a Google Cloud VPC is **global**, and subnets are **regional**. There is no per-AZ subnet the way there is in AWS. Traffic control uses **firewall rules** attached to the network with target tags or service accounts, rather than AWS-style security groups on the instance.
- **Cloud Load Balancing** — the global external HTTP(S) load balancer uses a single anycast IP that serves users from the nearest healthy backend. Also available: regional internal load balancers, and L4 network load balancers.
- **Cloud IAM** — the model is *who* (principal) can do *what* (role) on *which* resource. Roles are basic, predefined, or custom; prefer predefined roles at the narrowest scope. **Service accounts** are the identity that workloads use — the equivalent of an AWS IAM role for EC2 — and Workload Identity is how GKE pods assume them without key files.
- **Resource hierarchy** — Organisation → Folders → Projects → Resources. IAM policies inherit downwards. The **project** is the core billing and isolation boundary; there is no direct AWS equivalent, so mention it.
- **Cloud Storage** — object storage, with Standard, Nearline, Coldline and Archive classes, lifecycle rules, versioning and uniform bucket-level access. The equivalent of S3.
- **Databases** — **Cloud SQL** (managed MySQL, PostgreSQL, SQL Server), **AlloyDB** (PostgreSQL-compatible, higher performance), **Spanner** (horizontally scalable relational with strong consistency across regions), **Bigtable** (wide-column, very high throughput), **Firestore** (document), **BigQuery** (serverless analytics warehouse).
- **Pub/Sub** — global messaging for decoupling services; the rough equivalent of SNS plus SQS.
- **Cloud Logging and Cloud Monitoring** (the operations suite) — log-based metrics, uptime checks, alerting policies, dashboards, plus Error Reporting, Cloud Trace and Cloud Profiler for debugging in production.
- **Infrastructure as Code** — Deployment Manager reached end of support in March 2026, so the current answers are **Infrastructure Manager** or, far more commonly in industry, **Terraform** with the Google provider.

GCP troubleshooting scenarios worth rehearsing:

1. A GKE pod is stuck in Pending — how do you tell whether it is insufficient node capacity, a taint, or an unbound persistent volume claim?
2. A Compute Engine instance is unreachable on port 443 — how do you check firewall rules, network tags, the routing, and the service itself, in that order?
3. A Cloud Run service returns 429 or scales to an unexpectedly high instance count — where do you look?
4. Cloud SQL shows high replication lag — what are the likely causes and what do you check first?
5. A service account cannot write to a bucket — how do you debug an IAM permission problem using audit logs and the policy troubleshooter?

A useful thing to say in the interview: *"The concepts transfer almost completely between the two clouds — what changes is the naming, the resource hierarchy, and a few genuine design differences like Google Cloud's global VPC with regional subnets. For a multi-cloud estate the harder problems are the operational ones: consistent IAM, consistent monitoring, and the cost of moving data between providers."*

Part 7 — Databases

7.1 [L1] SQL fundamentals

- **ACID:** Atomicity (all or nothing), Consistency (valid state to valid state), Isolation (concurrent transactions don't interfere), Durability (committed data survives a crash).
- **Normalisation:** 1NF (atomic values), 2NF (no partial dependency on a composite key), 3NF (no transitive dependency), BCNF. **Denormalisation** trades redundancy for read speed — common at scale.
- **Keys:** primary, foreign, candidate, composite, unique, surrogate.
- **Joins:** INNER, LEFT, RIGHT, FULL OUTER, CROSS, SELF. Be able to draw them.
- **Indexes:** speed up reads, slow down writes, consume storage. Clustered (defines physical order, one per table) vs non-clustered. B-tree is the default structure.
- **DELETE vs TRUNCATE vs DROP:** DELETE is row-wise, logged, rollback-able, honours WHERE; TRUNCATE removes all rows quickly with minimal logging; DROP removes the table structure entirely.
- **WHERE vs HAVING:** WHERE filters rows before grouping, HAVING filters groups after aggregation.
- **UNION vs UNION ALL:** UNION removes duplicates — the database may do this by sorting or by hashing, depending on the engine and query plan — while UNION ALL does not, and is faster.

7.2 [L1] Queries you should be able to write on paper

Second-highest salary; Nth-highest salary; find duplicate rows; employees earning more than their department average; count per group with HAVING; self-join for manager-employee; window functions ROW_NUMBER(), RANK(), DENSE_RANK().

-- Second highest salary

```
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

-- Nth highest using a window function

```
SELECT DISTINCT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employees
) t WHERE rnk = 3;
```

-- Duplicates

```
SELECT email, COUNT(*) FROM users
GROUP BY email HAVING COUNT(*) > 1;
```

7.3 [L2] SQL vs NoSQL

Relational databases give you a fixed schema, strong consistency, joins, and ACID transactions — the right choice for bookings, payments, and anything where correctness beats throughput. NoSQL families — document (MongoDB), key-value (Redis, DynamoDB), column-family (Cassandra), graph (Neo4j) — give flexible schemas, horizontal scale, and high write throughput. Consistency varies by system: several offer tunable or strong consistency, so “NoSQL means eventual consistency” is an over-simplification.

CAP theorem: under a network Partition you must choose between Consistency and Availability. You cannot have all three simultaneously. As a broad simplification, traditional RDBMS clusters lean CP while Cassandra and DynamoDB lean AP — but both are configurable, so treat this as a rule of thumb rather than a fixed classification.

7.4 [L2] Reliability-relevant database topics

- **Replication:** master–slave (read scaling, failover), master–master (write availability, conflict risk), synchronous vs asynchronous (durability vs latency), **replication lag** and the stale-read problem it causes.
- **Sharding/partitioning:** horizontal split by key; watch for hot shards and cross-shard joins.
- **Connection pooling and connection exhaustion** — an extremely common production incident.
- **Slow queries:** find them in the slow query log, use EXPLAIN/EXPLAIN ANALYZE, look for full table scans, add or fix indexes, avoid SELECT *.
- **Deadlocks** in the database — detected and one transaction is rolled back; reduce by keeping transactions short and accessing tables in a consistent order.
- **Backups:** full vs incremental vs differential; **an untested backup is not a backup.** Point-in-time recovery. Define RTO and RPO.

7.5 [L3] Advanced

Isolation levels and the anomalies each permits (read uncommitted → dirty read; read committed → non-repeatable read; repeatable read → phantom read; serializable) · MVCC · two-phase commit · query plan caching · read/write splitting at the proxy layer · caching strategies (cache-aside, write-through, write-behind) and cache stampede.

Part 8 — DevOps Tooling

8.1 [L1] What DevOps actually is

A culture and practice that shortens the gap between writing code and running it in production reliably, through collaboration, automation, continuous integration and delivery, and feedback. **DevOps vs SRE:** DevOps describes the philosophy; SRE is one concrete implementation of it, defined by Google, that treats operations as a software problem and adds specific measures — SLOs, error budgets, toil limits.

8.2 [L1] Git

clone, add, commit, push, pull, fetch, branch, checkout/switch, merge, rebase, stash, reset, revert, cherry-pick, log, diff, tag.

- **merge vs rebase:** merge preserves history and creates a merge commit; rebase rewrites commits onto a new base for a linear history. **Never rebase a shared branch.**
- **reset vs revert:** reset moves the branch pointer (rewrites history — dangerous on shared branches); revert creates a new commit undoing the change (safe).
- **Branching strategies:** Git Flow, GitHub Flow, trunk-based development.

8.3 [L1] CI/CD

Pipeline stages: commit → build → unit tests → static analysis/security scan → package artifact → deploy to dev → integration tests → deploy to staging → acceptance/performance tests → approval → deploy to production → smoke tests → monitor.

- **Continuous Integration** — merge and test small changes frequently.
- **Continuous Delivery** — every build is deployable; release is a manual decision.
- **Continuous Deployment** — every passing build goes to production automatically.

Deployment strategies:

Strategy	How it works	Trade-off
Recreate	Stop old, start new	Simple; causes downtime
Rolling	Replace instances in batches	No downtime; two versions live at once
Blue-Green	Two identical environments, switch traffic	Instant rollback; doubles infrastructure cost
Canary	Small % of traffic to new version, then increase	Lowest blast radius; needs good monitoring
Feature flags	Deploy code dark, enable per user/segment	Very flexible; adds code complexity

Tools: Jenkins, GitLab CI, GitHub Actions, Azure DevOps, ArgoCD (GitOps), Spinnaker; artifacts in Nexus, Artifactory, or a container registry.

8.4 [L1] Containers

- **Image vs container:** an image is the immutable template; a container is a running instance of it.
- **Docker vs VM:** containers share the host kernel and isolate via namespaces and cgroups — lightweight, seconds to start. VMs each carry a full guest OS on a hypervisor — heavier, better isolated.
- **Dockerfile essentials:** FROM, RUN, COPY, ADD, WORKDIR, ENV, EXPOSE, CMD, ENTRYPOINT, HEALTHCHECK. Know CMD vs ENTRYPOINT (CMD is the default, overridable; ENTRYPOINT is the fixed executable).
- **Layers and caching:** each instruction creates a layer; order instructions so that rarely changing steps come first. **Multi-stage builds** shrink final images dramatically.
- **Persistence:** containers are ephemeral — use **volumes** for data.
- **Networking:** bridge, host, none, overlay.
- **Common commands:** docker build -t, run -d -p, ps -a, logs -f, exec -it, images, rmi, prune, compose up.

8.5 [L2] Kubernetes

- **Why:** container orchestration — scheduling, self-healing, service discovery, load balancing, rolling updates and rollbacks, autoscaling, secret and config management.
- **Control plane:** API server, etcd (cluster state), scheduler, controller manager. **Nodes:** kubelet, kube-proxy, container runtime.
- **Objects:** Pod (smallest deployable unit, one or more containers sharing network and storage), ReplicaSet, Deployment, StatefulSet, DaemonSet, Job/CronJob, Service (ClusterIP, NodePort, LoadBalancer), Ingress, ConfigMap, Secret, PV/PVC, Namespace, HPA.
- **Probes:** liveness (restart if failing), readiness (remove from service endpoints if failing), startup (for slow starters). Get this right — it is a favourite question and it is directly about reliability.
- **Debugging a pod:** kubectl get pods, describe pod (events at the bottom are usually the answer), logs -f, logs --previous, exec -it, get events --sort-by=.lastTimestamp, top pod.
- **Common failure states to be able to explain:** CrashLoopBackOff, ImagePullBackOff, Pending (unschedulable — insufficient resources or taints), OOMKilled, Evicted.
- **Resources:** requests drive scheduling; limits cap usage. Exceeding a memory limit gets the container OOMKilled; exceeding CPU throttles it.

8.6 [L2] Infrastructure as Code and configuration management

- **Terraform:** declarative, cloud-agnostic provisioning. init, plan, apply, destroy. Know **state file** (and why it must be remote and locked), modules, providers, variables and outputs, and **idempotency**.
- **Ansible:** agentless configuration management over SSH. Playbooks (YAML), inventory, roles, modules, handlers, idempotent tasks. Also Puppet and Chef (agent-based, pull model).

- **Terraform vs Ansible:** Terraform provisions infrastructure; Ansible configures what's on it. They complement rather than compete.
- **Why IaC:** version control, repeatability, peer review, disaster recovery, and a systematic way to prevent, detect and reconcile configuration drift — IaC reduces drift, it does not automatically eliminate it — and no snowflake servers.

8.7 [L2] Monitoring and observability

- **Monitoring** tells you *whether* something is broken (known unknowns); **observability** lets you ask *why* without shipping new code (unknown unknowns).
 - **Three pillars:** metrics, logs, traces.
 - **The four golden signals:** latency, traffic, errors, saturation. Memorise these four words — the question comes up often.
 - **USE method** (resources): utilisation, saturation, errors. **RED method** (services): rate, errors, duration.
 - **Stack:** Prometheus (pull-based metrics, PromQL) + Grafana (dashboards) + Alertmanager; ELK/EFK or Splunk for logs; Jaeger, Zipkin, or OpenTelemetry for traces; Datadog, New Relic, Dynatrace as commercial suites; PagerDuty/Opsgenie for on-call.
 - **Alerting hygiene: alert on symptoms, not causes.** Every alert must be actionable and must have a runbook. Alert fatigue is a genuine reliability risk — if engineers start ignoring alerts, your monitoring has failed.
-

8.8 [L1] Security fundamentals every SRE needs

Reliability and security are the same discipline seen from two angles — both are about a system behaving as intended under adverse conditions. Expect at least one security question.

- **Least privilege and service identity** — every human and every workload gets the minimum permission needed, scoped as narrowly as possible, and reviewed periodically. Workloads authenticate as roles or service accounts, never as a shared human account.
- **Never put credentials in source code** — no keys in Git, no passwords in Dockerfiles, no tokens in CI logs. Use a secrets manager (AWS Secrets Manager, Google Secret Manager, HashiCorp Vault) with rotation, injected at runtime. Scan repositories for accidentally committed secrets.
- **Encryption in transit and at rest** — TLS for every network hop including internal ones where feasible, disk and object-store encryption by default, and managed keys (KMS) with a documented rotation policy.
- **Certificate lifecycle** — expired certificates are one of the most common self-inflicted outages in the industry. Automate issuance and renewal, and alert well before expiry, not on the day.
- **Patch and vulnerability management** — a defined cadence for OS and dependency patching, container image scanning in the pipeline, base images rebuilt regularly, and a documented process for emergency patching of a critical vulnerability.
- **Network controls** — security groups and firewall rules that default to deny, private subnets for anything that does not need public exposure, bastion hosts or session managers instead of open SSH, and no databases reachable from the internet.
- **Audit logging** — who did what, when, and from where; logs shipped to a separate account or project so they cannot be tampered with by a compromised workload; retention that matches your compliance obligations.
- **Change control** — peer-reviewed infrastructure changes, no manual console changes in production, and an auditable trail from a code review to a deployment.

If asked "how do these relate to your job as an SRE?" — say that most of them are also reliability controls: a rotated certificate prevents an outage, a scanned image prevents an emergency rollout, an audit log shortens an incident investigation, and least privilege limits the blast radius of a mistake as much as of an attack.

Part 9 — SRE Core Concepts

This part is what differentiates you from candidates who prepared for a generic IT interview. Sabre is hiring an SRE, and the theory below is the vocabulary of the role.

9.1 [L1] What is SRE?

Site Reliability Engineering is the discipline of applying software engineering practices to infrastructure and operations problems, with the goal of building scalable and highly reliable systems. The founding idea, from Google: *"SRE is what happens when you ask a software engineer to design an operations team."*

Core practices: define reliability numerically with SLOs, spend an error budget deliberately, cap and automate toil, run blameless postmortems, share on-call ownership with developers, and gate launches on production readiness.

9.2 [L1] SLI, SLO, SLA — you will be asked, so be exact

- **SLI (Indicator)** — a measured number. “99.95% of flight-search requests returned successfully in the last 28 days.”
- **SLO (Objective)** — the internal target for that number. “99.9% availability over 28 days.”
- **SLA (Agreement)** — the contractual promise to a customer, with financial penalties. Typically looser than the SLO, so you have room to fix problems before you owe money.

Common SLIs: availability, latency (measure the **p95/p99**, never the mean), error rate, throughput, durability, freshness.

9.3 [L1] Error budget

If the SLO is 99.9% over a 30-day window, the allowed unavailability is 0.1% — about **43.2 minutes**. (The table below uses an average calendar month, which gives 43.8 minutes — same target, different window length.) That is the error budget. It reframes reliability as a resource to spend rather than a moral absolute:

- **Budget remaining** → ship features faster, take more risk.
- **Budget exhausted** → freeze risky releases, spend the cycle on reliability work.

This is what makes the developer–SRE relationship data-driven rather than political. Say that in the interview.

The nines, memorised:

Availability	Downtime per year	Per month	Per week
99%	3.65 days	7.2 hours	1.68 hours
99.9%	8.76 hours	43.8 minutes	10.1 minutes
99.95%	4.38 hours	21.9 minutes	5 minutes
99.99%	52.6 minutes	4.38 minutes	1 minute
99.999%	5.26 minutes	26 seconds	6 seconds

9.4 [L1] Toil — the JD names this explicitly

Toil is manual, repetitive, automatable, tactical work that scales linearly with service size and produces no enduring value. Restarting a service by hand every night is toil. Writing the automation that restarts it and fixes the root cause is engineering.

Google’s guidance is to keep toil under **50%** of an SRE’s time. If asked “*how would you reduce toil?*”: measure where the time goes → rank by frequency × time × risk → automate the top item → write a runbook for the rest → eliminate the root cause where possible → track the reduction.

9.5 [L1] Reliability metrics

- **MTTD** — Mean Time To Detect. Improved by better monitoring and alerting.
- **MTTA** — Mean Time To Acknowledge. Improved by good on-call rotation and paging.
- **MTTR** — Mean Time To Recover/Repair. **The JD explicitly asks you to reduce this.** Improved with runbooks, fast rollback, automation, good dashboards, and practice.
- **MTBF** — Mean Time Between Failures. Improved by redundancy, testing, and change quality.
- **RTO / RPO** — how fast you must recover, and how much data you can afford to lose.

Availability = $\text{MTBF} / (\text{MTBF} + \text{MTTR})$. You improve availability either by failing less often or by recovering faster — and recovering faster is usually the cheaper lever. That's a strong sentence to deliver.

9.6 [L1] Incident management — the JD's “high-pressure situations”

Lifecycle: detect → alert/page → triage and assign severity → assemble responders → **mitigate (restore service)** → resolve → root cause analysis → postmortem → action items → verify the fix.

Roles in a major incident: Incident Commander (decides, does not debug), Operations/Ops Lead (executes changes), Communications Lead (updates stakeholders and status page), Scribe (timeline).

Severity levels: typically Sev1 = total outage or critical business impact, all hands; Sev2 = major degradation, urgent; Sev3 = partial or minor impact; Sev4 = cosmetic or low.

The golden rule to say out loud: *“Mitigate first, diagnose later. Restoring service takes priority over finding the root cause — but preserve logs, metrics and a snapshot before you restart anything, or you lose the evidence.”* That caveat is the mark of someone who has actually thought about it.

Blameless postmortem: documents timeline, impact, detection, root cause, resolution, what went well, what went badly, and dated action items with owners. Blameless does not mean human actions are ignored — it means they are examined alongside the systemic conditions that made the error likely, without punishment or personal blame. If you punish individuals, people hide information, and you lose the ability to learn. Every incident is a free lesson you already paid for.

5 Whys is the simplest RCA technique: keep asking why until you reach a systemic cause rather than a human one.

9.7 [L2] Reliability design patterns

- **Redundancy and no single point of failure** — N+1, multi-AZ, multi-region.
- **Graceful degradation** — serve reduced functionality rather than an error page. For a travel site: if the personalised-recommendation service is down, still show search results.
- **Circuit breaker** — stop calling a failing dependency so you don't cascade the failure; retry after a cooldown.

- **Retries with exponential backoff and jitter** — retries without backoff amplify an outage into a self-inflicted DDoS. Jitter prevents synchronised retry storms.
- **Timeouts everywhere** — an unbounded wait exhausts your thread and connection pools.
- **Rate limiting and load shedding** — protect the core by rejecting excess work early.
- **Bulkheads** — isolate resource pools so one failing dependency cannot consume everything.
- **Idempotency** — makes safe retries possible; essential for bookings and payments.
- **Chaos engineering** — deliberately inject failure in controlled conditions to prove your assumptions. Netflix's Chaos Monkey is the canonical example.
- **Capacity planning** — forecast peak load, load test to find the breaking point, keep headroom.

9.8 [L2] On-call

Rotation, escalation policy, paging thresholds, handover notes, runbooks, and follow-the-sun coverage across time zones. **A good answer when asked if you're comfortable with on-call:** yes, with a caveat that shows judgment — *"Yes. I'd expect a proper rotation, documented runbooks, and that we treat repeated pages as a bug to fix rather than a routine to endure."*

9.9 [L3] Advanced

Error budget policy enforcement · production readiness reviews before launch · SLO burn-rate alerting (fast vs slow burn windows) · request hedging · progressive delivery · disaster recovery tiers (backup-restore, pilot light, warm standby, active-active) · game days and DR drills · cost of reliability vs marginal benefit — the argument that chasing an extra nine is sometimes the wrong business decision.

Part 10 — Scripting & Automation

The JD says “reduce toil by automating manual processes.” That means scripting. You will not be asked to build a compiler — you will be asked to parse a log, check a service, and clean up disk space.

10.1 [L1] Bash

Know: shebang `#!/bin/bash`, variables, `$1` `$2` `$@` `$#`, `$?` (exit status), conditionals `if [ ] then fi`, comparison operators (`-eq` `-ne` `-gt` `-lt`, `-f` `-d` `-z`), loops (`for`, `while`), functions, `case`, redirection (`>` `>>` `2>&1`), pipes, cron syntax, `trap`, `set -euo pipefail`.

```
#!/bin/bash
# Alert if any filesystem exceeds a threshold
THRESHOLD=80
df -h | awk 'NR>1 {print $5, $6}' | while read -r usage mount; do
    used=${usage%\%}
    if [ "$used" -ge "$THRESHOLD" ]; then
        echo "WARNING: $mount is at ${used}% used"
    fi
done

#!/bin/bash
# Restart a service if its health check fails
if ! curl -sf http://localhost:8080/health > /dev/null; then
    echo "$(date) health check failed, restarting" >> /var/log/watchdog.log
    systemctl restart myapp
fi
```

Do not ship this as written. A single failed health check is not enough evidence to restart a production service — a transient blip will cause a restart loop. Add a consecutive-failure threshold, a backoff between checks, a cap on restarts per hour, and an alert whenever the watchdog acts. A watchdog that restarts silently hides the real fault.

```
# Top 10 IPs by request count in an access log
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -10

# Count HTTP 5xx responses today
grep "$(date +%d/%b/%Y)" access.log | awk '$9 ~ /^5/ {c++} END {print c+0}'

# Delete log files older than 30 days
find /var/log/myapp -name "*.log" -type f -mtime +30 -print
```

Note the dry run. The command above only prints what it would remove. Add `-delete` once you have read the output, always include `-type f` so directories are never matched, and prefer controlled retention through `logrotate` over ad-hoc deletion in production.

Cron format: minute hour day-of-month month day-of-week command. `0 2 * * *` = every day at 02:00. `* / 5 * * * *` = every five minutes.

10.2 [L1] Python for ops

Know: file I/O with context managers, string handling, lists/dicts/sets, list comprehensions, functions, exception handling, os, sys, subprocess, json, re, datetime, requests, logging, argparse.

```
import re
from collections import Counter

pattern = re.compile(r'(\d+\.\d+\.\d+\.\d+).*?" (\d{3}) ')
errors = Counter()

with open('access.log') as f:
    for line in f:
        m = pattern.search(line)
        if m and m.group(2).startswith('5'):
            errors[m.group(1)] += 1

for ip, count in errors.most_common(5):
    print(f'{ip}: {count} server errors')

import requests, sys

ENDPOINTS = ["https://api.example.com/health", "https://web.example.com/health"]

for url in ENDPOINTS:
    try:
        r = requests.get(url, timeout=5)
        status = "UP" if r.status_code == 200 else f"DEGRADED ({r.status_code})"
    except requests.RequestException as e:
        status = f"DOWN ({e.__class__.__name__})"
    print(f'{url}: {status}')
```

10.3 [L2] DSA — how much you actually need

For an SRE fresher role, expect easy to lower-medium difficulty, if coding is tested at all. Cover:

- **Arrays and strings:** two pointers, sliding window, reverse, palindrome, anagram, max subarray (Kadane), missing number, two-sum.
- **Hashmaps:** frequency counting, first non-repeating character, duplicate detection. This is the single highest-yield structure for ops-style problems.
- **Sorting and searching:** binary search and its variants; know the complexity of the common sorts.
- **Stacks and queues:** balanced parentheses, next greater element.
- **Linked lists:** reverse, detect cycle (Floyd's), find the middle.
- **Recursion basics:** factorial, Fibonacci with memoisation.
- **Complexity analysis:** be able to state and justify time and space complexity for anything you write.

Trees, graphs and dynamic programming are worth a light pass, but do not spend your five days there at the cost of Linux and cloud.

Part 11 — Your Resume & Projects

11.1 The rules

- One page. Every line must be defensible — assume every keyword will be questioned.
- Remove anything you cannot discuss for two minutes. A weak keyword is worse than a missing one.
- Quantify: “reduced deployment time from 20 minutes to 4” beats “improved deployment process.”
- Match the JD’s vocabulary honestly — cloud, Linux, CI/CD, automation, monitoring.

11.2 If you have no SRE experience (most of you)

Build one small project this week and put it on your resume honestly. Any of these is enough to talk about:

1. **Containerise and deploy an app:** write a Dockerfile for a simple web app, push to a registry, run it on a free-tier VM, put a reverse proxy in front.
2. **Build a CI/CD pipeline:** GitHub Actions that runs tests on push, builds an image, and deploys on merge to main.
3. **Monitoring stack:** Prometheus + Grafana + node_exporter on a VM; build a dashboard and one alert rule.
4. **Automation script:** a health-check and log-rotation script running on cron, with alerting.
5. **IaC:** a small Terraform config that provisions a VPC and a VM.

Then be able to say what broke and how you fixed it. That story is worth more than the project itself — it’s evidence you can debug, which is the whole job.

11.3 Project answer template (STAR)

- **Situation** — what was the context and why did it matter?
- **Task** — what were *you* responsible for?
- **Action** — what did you do, with which tools, and what did you choose *not* to do and why?
- **Result** — what changed, ideally with a number, and what did you learn?

Keep it to 90 seconds. Then stop talking and let them ask.

Part 12 — The Question Bank

Practise these **out loud**. Reading answers creates the illusion of readiness; speaking them exposes the gaps.

12.1 Linux (25)

1. What happens when you boot a Linux machine?
2. Explain the Linux directory structure — /etc, /var, /proc, /opt, /tmp, /usr.
3. What does chmod 755 mean, and when would you use it?
4. Difference between hard link and soft link.
5. How do you find the 10 largest files on a filesystem?
6. What is the difference between kill, kill -9, and kill -15?
7. How do you check which process is listening on port 8080?
8. What is a zombie process? An orphan process?
9. Explain load average. Is a load of 4 bad?
10. What is swap, and is swap usage always a problem?
11. Difference between su and sudo.
12. How do you schedule a job to run every 5 minutes?
13. What is /etc/passwd vs /etc/shadow?
14. How do you check disk usage vs inode usage, and why does the difference matter?
15. What is an inode?
16. Explain the SUID, SGID and sticky bits.
17. How do you troubleshoot a server that is running slowly?
18. What does grep -v do? grep -r? grep -E?
19. How do you find and delete files older than 7 days?
20. Difference between soft mount and hard mount.
21. What is systemd and how does it differ from init?
22. How do you check the last 100 lines of a log and follow it live?
23. What is the OOM killer and when does it act?
24. How do you check open file descriptors for a process?
25. Explain the difference between a process and a thread.

12.2 Networking (20)

1. Explain the OSI model with an example for each layer.
2. TCP vs UDP — with two real use cases each.
3. Explain the TCP three-way handshake.
4. What happens when you type a URL and hit Enter?
5. How does DNS resolution work end to end?
6. What is the difference between 401 and 403? 502 and 504?
7. What is a load balancer and how does L4 differ from L7?
8. What is NAT and why is it needed?
9. What is ARP?

10. Explain subnetting — what does /24 mean?
11. What is the difference between a switch, a router and a hub?
12. How does HTTPS work? Explain the TLS handshake.
13. What is a CDN and how does it reduce latency?
14. How would you troubleshoot “the website is unreachable”?
15. What is a reverse proxy vs a forward proxy?
16. What are the well-known ports for SSH, HTTP, HTTPS, DNS, MySQL, PostgreSQL, SMTP?
17. What is TTL in DNS and why does it matter during migration?
18. What causes packet loss and how would you diagnose it?
19. What is the difference between latency, bandwidth and throughput?
20. What is a VPN and why would a company need one?

12.3 Cloud (25)

1. What is cloud computing and what problems does it solve?
2. IaaS vs PaaS vs SaaS with examples.
3. Public vs private vs hybrid vs multi-cloud.
4. Explain the shared responsibility model.
5. Region vs Availability Zone — how do you use each for resilience?
6. Vertical vs horizontal scaling — which do you prefer and why?
7. What is auto-scaling and how does it decide to scale?
8. Explain EC2 instance purchasing options and when to use spot.
9. What is S3 and what are its storage classes?
10. Difference between S3 and EBS.
11. Security groups vs NACLs.
12. What is a VPC? Why separate public and private subnets?
13. What is IAM? What is a role vs a user? Why prefer roles?
14. What is the principle of least privilege?
15. What is an internet gateway vs a NAT gateway?
16. How would you design a highly available web application on AWS?
17. What are the 7 R's of cloud migration?
18. How would you migrate a legacy datacenter app to the cloud? Walk through your plan.
19. What are the risks in a migration and how do you mitigate them?
20. Why would a company use multi-cloud? What does it cost them?
21. Name the GCP equivalents of EC2, S3, RDS, CloudWatch and EKS.
22. What is serverless and when is it the wrong choice?
23. How do you monitor and control cloud spend?
24. What is disaster recovery in the cloud? Explain RTO and RPO.
25. What is a CDN and how does it fit into a cloud architecture?

12.4 DevOps (25)

1. What is DevOps and what problem was it invented to solve?

2. How does SRE differ from DevOps?
3. Explain a CI/CD pipeline end to end.
4. Continuous delivery vs continuous deployment.
5. What is a build artifact and where do you store it?
6. Explain blue-green, canary, and rolling deployments with trade-offs.
7. How do you roll back a bad deployment?
8. Git merge vs rebase — when do you use each?
9. Git reset vs revert.
10. How do you resolve a merge conflict?
11. What is a Dockerfile? Explain CMD vs ENTRYPOINT.
12. Docker image vs container.
13. How are containers different from virtual machines?
14. How do you persist data in a container?
15. How would you reduce a Docker image's size?
16. What is Kubernetes and what problem does it solve?
17. Explain pod, deployment, service and ingress.
18. Liveness vs readiness vs startup probes.
19. What does CrashLoopBackOff mean and how do you debug it?
20. What are resource requests and limits?
21. What is Infrastructure as Code and why does it matter?
22. Terraform vs Ansible.
23. What is the Terraform state file and why must it be remote?
24. What does idempotent mean and why does it matter in automation?
25. Which monitoring tools have you used, and what would you alert on?

12.5 SRE-specific (25)

1. What is Site Reliability Engineering?
2. Why do you want to be an SRE and not a developer? (*Expect this. Prepare it properly.*)
3. Define SLI, SLO and SLA with an example for a flight booking API.
4. What is an error budget and how would you use it in a disagreement with a product team?
5. What is toil? Give an example and explain how you'd eliminate it.
6. What are the four golden signals?
7. Monitoring vs observability.
8. Why alert on symptoms rather than causes?
9. What is MTTR and name four concrete ways to reduce it.
10. How much downtime does 99.9% availability allow per month?
11. Walk me through how you'd handle a Sev1 outage.
12. What is a blameless postmortem and why blameless?
13. What is a runbook and what belongs in a good one?
14. What is a circuit breaker pattern?
15. Why do retries need exponential backoff and jitter?

16. What is graceful degradation? Give a travel-industry example.
17. What is chaos engineering and why deliberately break production?
18. How do you plan capacity for a predictable traffic spike?
19. What is a single point of failure and how do you find one?
20. How would you reduce alert fatigue on a noisy service?
21. What would you do in your first 90 days owning a service you didn't build?
22. How do you decide whether to roll back or fix forward?
23. What is idempotency and why does it matter for retries?
24. How do you balance reliability work against feature delivery?
25. Is 100% availability the right target? Defend your answer.

12.6 Scenario questions (15) — where L2 candidates are separated

1. Production is down and you're the first responder. What are your first five minutes?
2. Users report the site is slow but all your dashboards are green. What now?
3. A deployment went out 20 minutes ago and errors are climbing. What do you do?
4. Disk on a production database server is at 98%. Walk me through it.
5. CPU on one web server is pinned at 100% while the other three are idle. What's likely wrong?
6. A service works fine in staging but fails in production. How do you approach it?
7. You're paged at 3am for an alert that has fired every night for a week. What do you do — short term and long term?
8. A dependency you call has become very slow. How do you stop it taking your service down?
9. Traffic is 10× normal because of a fare sale. What breaks first, and what do you do?
10. Someone deleted a production database table. Walk through your response.
11. You need to migrate a legacy app to AWS with less than 5 minutes of downtime. How?
12. Two teams blame each other during an incident. You're the incident commander. What do you do?
13. Your error budget is exhausted and the product team wants to ship a big feature. How do you handle the conversation?
14. Certificates expired in production overnight. What happened, and how do you prevent recurrence?
15. Logs show intermittent 504s from one of six backend instances. How do you isolate it?

For every scenario, use this structure: clarify the scope and impact → check monitoring and recent changes → form a hypothesis → mitigate to restore service → verify → root cause → prevent recurrence → communicate throughout. Interviewers are grading your *method*, not whether you guess the right cause.

12.7 Sample model answers for the four highest-stakes questions

“Why SRE and not development?”

“I enjoy building, but what I actually found most engaging in my projects was the part after the code was written — getting it deployed reliably, figuring out why it broke,

and automating so it wouldn't break the same way twice. In my [project], I spent more time on the deployment pipeline than the application, and I preferred it. SRE is that work as a discipline, with software engineering applied to it, so I'm not choosing between coding and operations — I'd be writing code to solve operational problems. And at a company like Sabre where systems are 24×7 and migrating to multi-cloud, that work has real consequences."

"You have no production experience. Why should we hire you?"

"That's fair — I haven't run production systems. What I do have is the fundamentals: Linux, networking, cloud, and enough scripting to automate. I've built [project] which taught me how much I don't know, and I'm comfortable saying 'I don't know, here's how I'd find out.' For an entry-level Site Reliability Engineer I candidate, I think what matters is whether I can debug systematically under pressure, learn quickly, and follow procedure carefully — and I'd point to [specific example] as evidence."

"Are you comfortable with on-call and high-pressure incidents?"

"Yes. I don't want to pretend I know what a 3am Sev1 feels like, but I understand the shape of it — stay calm, restore service first, communicate clearly, don't make uncoordinated changes, and write it up honestly afterwards. I'd also expect to spend my early months learning the runbooks properly so that when I am on call, I'm useful rather than a bystander."

"Where do you see yourself in five years?"

"Still in reliability engineering, but deeper — owning larger systems, doing more of the design and automation and less of the manual triage, and ideally mentoring newer engineers through incidents. I'm interested in the multi-cloud migration side specifically, so building real depth there over the next few years is what I'd want."

Note what that answer does **not** say: it does not mention switching to development, MBAs, or higher studies.

Part 13 — HR & Behavioural Round

13.1 The standard set

Tell me about yourself · Why Sabre · Why SRE · Strengths and weaknesses · Biggest achievement · Biggest failure and what you learnt · A time you worked in a team · A time you handled conflict · A time you handled pressure or a tight deadline · A time you failed and recovered · How do you handle criticism · How do you prioritise when everything is urgent · Are you willing to relocate to Bengaluru · Are you willing to travel · Are you comfortable with shifts and on-call · Do you have any offers · Any questions for us?

13.2 The four the JD specifically sets up

The JD names **multi-tasking, English communication, willingness to travel, and attention to detail**. These will be probed. Have a concrete story for each:

- **Multi-tasking:** a time you handled competing priorities — and say *how* you prioritised, not just that you coped.
- **Attention to detail:** a time a small thing you caught prevented a bigger problem.
- **Travel and relocation:** answer with a clean yes if it's true. Hesitation reads as a flag.
- **Communication:** demonstrated by how you answer everything else. Structure your answers, don't ramble, and stop when you've finished.

13.3 “Tell me about yourself” — a working structure

Present → Past → Future, in 60–90 seconds.

“I’m a final year [branch] student at [college] with a [CGPA] CGPA. Over the last year I’ve focused on infrastructure and cloud — I’ve worked with Linux, AWS, Docker, and built [project], where I [specific thing]. Before that my interest was general software development, but working on deployment and debugging is what pulled me towards reliability engineering. Going forward I want to build a career in SRE, which is why this role at Sabre is a direct fit — production ownership, cloud migration and automation are exactly the areas I’ve been building towards.”

13.4 Weakness — how to answer without self-sabotage

Pick a real, non-critical weakness, and show the correction in progress. Avoid “I’m a perfectionist” and “I work too hard” — interviewers have heard them a thousand times and they read as evasive.

“I tend to go too deep into a problem before asking for help — in my project I spent almost two days on a deployment issue that a senior would have spotted in ten minutes. I’ve started setting myself a time limit: if I’m stuck for an hour, I ask, with a clear summary of what I’ve already tried. That’s made a real difference.”

Note that this weakness also demonstrates persistence and self-correction — both useful in an SRE.

13.5 Questions to ask them

Always have three. Good options:

- What does the on-call rotation look like for a new SRE, and how long before someone joins it?
- Which applications are currently being migrated, and how far along is the AWS/GCP journey?
- What does the first six months of ramp-up look like for an SRE I?
- What tooling does the team use for monitoring and incident management?
- What separates a good SRE from a great one on your team?

Never open with salary, work-from-home, or promotion timelines. Save compensation questions for the offer conversation.

13.6 Practical checklist

Documents: resume copies, marksheets all semesters, ID proof, passport-size photos, offer letters if any.

Appearance: formals, neat, arrive 30 minutes early.

During the interview: listen fully before answering · think out loud during technical problems · if you don't know, say so and describe how you'd find out — never bluff, because a good interviewer will keep pulling the thread · ask clarifying questions in scenarios · maintain eye contact · don't badmouth anyone · thank the interviewer.

The most valuable sentence you can learn: *"I don't know that, but here's how I'd approach finding out."* In SRE, admitting uncertainty is a safety property, not a weakness. Bluffing in production causes outages, and interviewers know it.

Part 14 — Mock Interview Structure

Run this twice with a peer before the drive. One of you plays interviewer with this handbook open.

Round 1 — Technical fundamentals (45 min)

- 5 min: tell me about yourself, resume walkthrough
- 15 min: Linux commands and troubleshooting
- 10 min: networking fundamentals
- 10 min: OS or DBMS concepts
- 5 min: a scripting question

Round 2 — Cloud, DevOps and scenarios (45 min)

- 10 min: cloud fundamentals and service knowledge

- 10 min: a migration or architecture design question
- 10 min: CI/CD, Docker, Kubernetes
- 10 min: two incident scenarios
- 5 min: SRE concepts — SLO, error budget, toil

Round 3 — Managerial / HR (30 min)

- Behavioural questions, the “why SRE” question, willingness to travel and relocate, on-call comfort, your questions for them.

Score each round on: correctness · structure of the answer · clarity of speech · honesty when unsure · the debugging method shown. Write down every question you fumbled and fix it the same day.

14.4 Practical readiness assessment — do this, do not just read it

A question bank proves recall. This proves capability. Each student should complete all six items and be able to demonstrate them in under thirty minutes to a trainer or peer. Tick each only when you have done it yourself, end to end.

1. **Linux troubleshooting** — on a live machine, find the largest directory, identify the top CPU and memory consumer, find which process holds a given port, and read the last 100 lines of a service log. Narrate what each command tells you.
2. **Dockerise an application** — write a Dockerfile for a simple web app, build it, run it with a published port and a mounted volume, and explain one change you made to reduce the image size.
3. **A CI/CD workflow** — a pipeline (GitHub Actions, GitLab CI or Jenkins) that runs on push, executes a test, builds the image and would deploy on merge. Show a run that failed and the run that fixed it.
4. **A cloud deployment** — deploy the container to a free-tier VM or managed container service on AWS or Google Cloud, reachable over the network, with the security group or firewall rule configured deliberately rather than left open.
5. **Monitoring and one alert** — expose or scrape a metric, build one dashboard panel, and configure one alert rule that actually fires when you break the service. Explain why you chose that alerting threshold.
6. **A five-minute incident simulation** — a peer breaks something in your environment without telling you what. You have five minutes to detect, communicate impact, mitigate, and state the next step. You are scored on method and communication, not on guessing the cause.

Scoring guidance for trainers: for each item, mark Not attempted / Completed with help / Completed independently / Can explain the trade-offs. A student at “can explain the trade-offs” on four or more items is genuinely interview-ready for an SRE I role.